MILESTONE-2 PROJECT AND ANALYSIS

BY-HRISHIRAJ SAHU

Analysis-1:

```
--Query-1
SELECT
payment_type,
cast(round(avg(payment_value)) as INTEGER) as avg_payment_value
from payments
group by payment_type
order by avg_payment_value;
```

Query-1:

To analyse the average payment value of each payment type.

Analysis:

According to the analysis credit card has the most average payment value compared to other payment types

Recommendation:

So for this the company has to track the regular payment type transaction and look out for the payment failures or issue problems.

```
--Query-2
SELECT payment_type,
round((count(order_id)*100)/sum(count(order_id)) over(),1) as order_percentage
from payments
group by payment_type
order by order_percentage desc;
```

Query-2:

To analyse the percentage of total orders for each payment type.

Analysis:

In the analysis of percentage of total orders the credit card still holds the top position when sorted in descending order and the customers are more in favour of this.

Recommendation:

According to the analysis of this the company has to track the payment behaviour and regular maintainance of the payment server for efficient transaction.

```
select p.product_id,o.price
from product p join order_items o on p.product_id=o.product_id
where p.product_category_name='smart' and o.price between 100 and 500 order by o.price desc;
```

Query-3:

To categorise the product category “smart” in the price range of 100 to 500 BRL

Analysis:

On the basis of analysis the highest sale in the “smart” category product is 439 BRL.

Recommendation:

On the basis of analysis the company should focus on the type of products category according to the needs and their requirements.

```sql
--Query-4
select TO_CHAR(o2.order_purchase_timestamp,'fmmonth') as month,
round(sum(o1.price+o1.freight_value),0) as total_sales
from orders o2 join order_items o1 on o2.order_id=o1.order_id
group by month
order by total_sales desc limit 3;
```

Query-4:

Analysis of top 3-months with the highest total sales value.

Analysis:

After analysing the top 3 months the may has the highest total sales value and July has the least sales value.

Recommendation:

So after the analysis the recommendation is that the company should understand the seasonal pattern behaviour of customer spending in which season there is no demand.

```sql
--Query-5
select p.product_category_name,(max(o.price)-min(o.price)) as price_difference
from product p join order_items o on p.product_id=o.product_id
group by p.product_category_name having (max(o.price)-min(o.price))>500
order by price_difference desc;
```

Query-5:

To identify the categories where the difference between maximum and minimum product price greater than 500 BRL.

Analysis:

On the basis of analysis there are 57 product categories which has price greater than the difference of maximum and minimum product price.

Recommendation:

So after the analysis the recommendation is that the company should track the demand of product categories month wise and

also the demand of least demanding products why they the least demanding.

```
--Query-6
select payment_type,stddev(payment_value) as std_deviation from payments
group by payment_type order by std_deviation;
```

Query-6:

To check the consistency of the transaction of payment type by checking least variance in payment types.

Analysis:

So according to the analysis the voucher has the smallest standard deviation value and customers are preferring vouchers more than the other payment types

Recommendation:

So the sole focus of the company to evaluate the other transaction types and why they are least consistent.

```
--Query-7
select product_id,product_category_name from product
where product_category_name isnull or length(trim(product_category_name))=1;
```

Query-7:

List of products which has missing category name or contain a single character.

Analysis:

There are 614 null product categories including the single character in this.

Analysis-2:

```sql
--Query-1
with OrderTotals as (

    select
        order_id,
        sum(payment_value) as total_value
    from
        payments
    group by
        order_id
),
Segments as (

    Select
        order_id,
        case
            when total_value < 200 then 'Low (< 200)'
            when total_value between 200 and 1000 then 'Medium (200-1000)'
            else 'High (> 1000)'
        end as order_value_segment
    from
        OrderTotals
```

```sql
Select
    s.order_value_segment,
    p.payment_type,
    Count(*) as count
from
    payments p
join
    Segments s on p.order_id = s.order_id
group by
    s.order_value_segment,
    p.payment_type
order by
    case
        when s.order_value_segment = 'Low (< 200)' then 1
        when s.order_value_segment = 'Medium (200-1000)' then 2
        else 3
    end asc,
    count desc;
```

Query-1:

To calculate the count of each payment type in three different order value segments(Low,Medium,High).

Analysis:

In three different segments the credit card has highest count of payments.

Recommendation:

So based on the analysis the company has to monitor the other categories of transaction to where the problem arises what type of problems customers are facing through these payments.

```sql
--Query-2
select p.product_category_name,min(o.price),max(o.price),avg(o.price)as avg_price
from product p join order_items o on p.product_id=o.product_id
group by p.product_category_name
order by avg_price desc;
```

Query-2:

To calculate the minimum and maximum and average price of each category.

Analysis:

After the analysis the “pcs” category has the highest category price in all three price ranges.

```sql
--Query-3
select c.customer_unique_id,count(order_id) as total_orders
from customers c join orders o on c.customer_id=o.customer_id
group by c.customer_unique_id having count(o.order_id)>1 order by total_orders desc;
```

Query-3:

To analyse the customers who placed multiple orders over time.

Analysis:

According to the analysis the most number of orders by a customer is 16.

Recommendation:

After analysis the company target those customers who have less orders from the site and try to understand their preferences.

```sql
--Query-4
create temporary table customer_order_summary as
select c.customer_unique_id,count(o.order_id)as order_qty
from customers c join orders o on c.customer_id=o.customer_id group by customer_unique_id;

select customer_unique_id,
case
 when order_qty=1 then 'new'
 when order_qty between 2 and 4 then 'Returning'
 else 'loyal'
 end as customer_type
from customer_order_summary
```

Query-4:

To analyse the customer order quantity into different types(New,Returning,Loyal).

Analysis:

After analysis there are large number of customers in the “new” category.

Recommendation:

So after analysis the company should focus on the customers order pattern and to checking the frequency of their order by how often they order things.

```sql
--Query-5
select p.product_category_name, sum(o.price+o.freight_value) as total_revenue
from product p join order_items o on p.product_id=o.product_id
group by p.product_category_name order by total_revenue desc limit 5;
```

Query-5:

To identify product categories which generates highest revenue.

Analysis:

After the analysis the product category which has higest revenue is “Beleza saude”.

Recommendation:

On the basis of analysis the recommendation is that the company should focus on the product category which has least revenue so they have to focus on improving that.

Analysis-3:

```sql
--Query-1
select * from orders;
select * from order_items;

select season,sum(order_total) as total_sales
from(
select sum(o2.price) as order_total,
case
  when extract(month from o1.order_purchase_timestamp) in (3,4,5) then 'Spring'
  when extract(month from o1.order_purchase_timestamp) in (6,7,8) then 'Summer'
  when extract(month from o1.order_purchase_timestamp) in (9,10,11) then 'Autumn'
  else 'Winter'
 end as season
from orders o1 join order_items o2
on o1.order_id=o2.order_id
group by o1.order_purchase_timestamp
)
group by season
order by season;
```

Query-1:

To compare the total sales pattern between different seasons.

Analysis:

So on the basis of analysis the spring season has the higest total sales value compare to the other season.

Recommendation:

After the analysis the recommendation is that company should try to understand the seasonal order pattern of customers and try to focus on the most demanding season of order.

```sql
--Query-2
select product_id,sum(product_photos_qty) as total_quantity_sold
from product
group by product_id having sum(product_photos_qty)>
(select avg(product_photos_qty) from product);
```

Query-2:

To identify products that have sales volume above the overall average.

Analysis:

There are approximately 96000 products which has sales volume greater than overall average.

```sql
--Query-3
select extract(month from o1.order_purchase_timestamp) as month,sum(o2.price+o2.freight_value)
as total_revenue from orders o1 join order_items o2 on o1.order_id=o2.order_id
where extract(year from o1.order_purchase_timestamp)=2018
group by month order by month;
```

Query-3:

To analyse the revenue generated in each month of 2018.

Analysis:

According to the anlalysis the least revenue month in 2018 is september.

Recommendation:

After the analysis the company try to focus on the least revenue generated month why there is declining in orders growth.

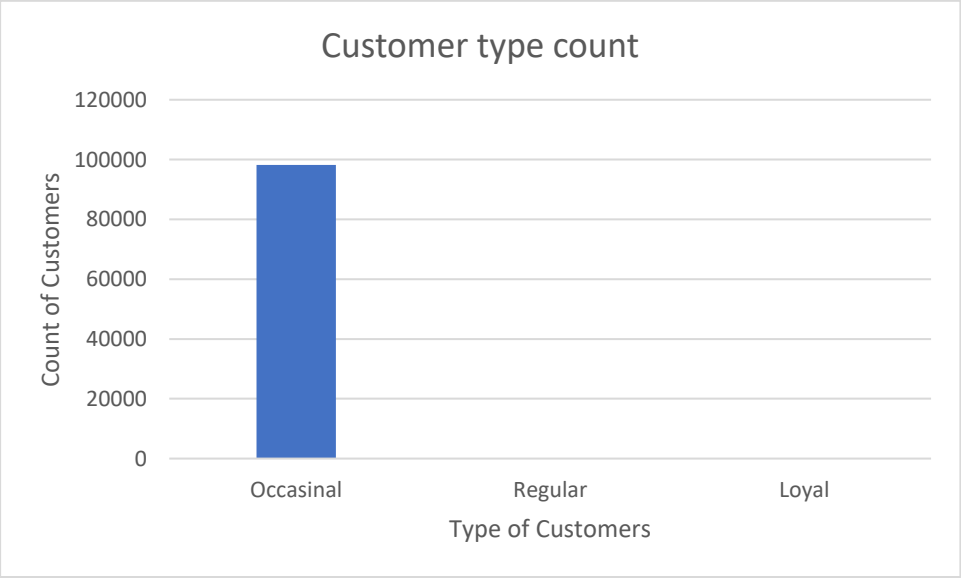
```sql
--Query-4
with total_orders as
(
select customer_id,count(order_id) as order_count from orders group by customer_id
),
customer_segmentation as
(
select customer_id,
case
 when order_count between 1 and 2 then 'Occasinal'
 when order_count between 3 and 5 then 'Regular'
 else 'Loyal'
 end as customer_type from total_orders
)
select customer_type,count(*) as count from customer_segmentation
group by customer_type
order by count desc;
```

Query-4:

To create segmentation based on purchase frequency of customers.

Analysis:

According to the analysis the count of occasional customer count are more than the other segmentation.



Recommendation:

So on the basis of analysis the recommendation should be to focus regular and loyal customers because there needs to be some improvement.

```
--Query-5
select o1.customer_id as customer_id,(sum(o2.price+o2.freight_value)/count(o1.order_id)) as
average_order_value,dense_rank() over(partition by
(sum(o2.price+o2.freight_value)/count(o1.order_id))) as customer_rank
from orders o1 join order_items o2 on o1.order_id=o2.order_id
group by o1.customer_id limit 20;
```

Query-5:

To identify customers with high ranking for exclusive rewards.

Analysis:

After the analysis in the top 20 customers are in the rank 1 value.

```
--Query-6
with recursive
indexed_sales as (
    Select
        oi.product_id,
        to_char(o.order_purchase_timestamp, 'YYYY-MM') as sale_month,
        sum(oi.price) as monthly_revenue,
        row_number() over (
            partition by oi.product_id
            order by to_char(o.order_purchase_timestamp, 'YYYY-MM')
        ) as rn
    from order_items oi
    join orders o on oi.order_id = o.order_id
    group by 1, 2
),


cumulative_sales as (

    Select
        product_id,
        sale_month,
        monthly_revenue as total_sales,
        rn
    from indexed_sales
    where rn = 1

    union all
```

```sql
    Select
        i.product_id,
        i.sale_month,
        c.total_sales + i.monthly_revenue,
        i.rn
    from indexed_sales i
    join cumulative_sales c on i.product_id = c.product_id and i.rn = c.rn + 1
)
Select
    product_id,
    sale_month,
    ROUND(cast(total_sales AS NUMERIC), 2) as total_sales
from cumulative_sales
order by product_id, sale_month;
```

Query-6:

To calculate monthly cumulative sales for each product.

Analysis:

According to the analysis there is not much difference in the growth of sale in their key product.

```sql
--Query-7- to calculate monthly sales payment for each payment type and compute the percentage
--change from previous month
with monthly_sales as (
    Select
        p.payment_type,
        date_trunc('month', o.order_purchase_timestamp) AS sale_month,
        sum(p.payment_value) AS monthly_total
    from orders o
    join payments p
       on o.order_id = p.order_id
    where o.order_purchase_timestamp >= DATE '2018-01-01'
      and o.order_purchase_timestamp <  DATE '2019-01-01'
    group by
        p.payment_type,
        DATE_TRUNC('month', o.order_purchase_timestamp)
),

monthly_growth AS (
    Select
        payment_type,
        sale_month,
        monthly_total,
        lag(monthly_total) over (
            partition by payment_type
            order by sale_month
        ) as previous_month_total
    from monthly_sales
)
```

```sql
Select
    payment_type,
    sale_month,
    monthly_total,
    round(
        (monthly_total - previous_month_total) * 100.0
        / nullif(previous_month_total, 0),
        2
    ) as monthly_change
from monthly_growth
order by
    payment_type,
    sale_month;
```

Query-7:

To calculate monthly sales payment for each payment type and compute the percentage change from previous month.

Analysis:

In the payment type monthly change there is not a significant amount of change in percentage from the previous month.